Programming languages

04 / Syntax and meaning

Lexical analysis and parsing

Lexical analysis groups characters into tokens such as identifiers, keywords and operators. Parsing checks the token sequence against a grammar and builds a syntax tree. In total = 3 + count, the lexer identifies the names and operators; the parser represents assignment and addition as structured expressions.

Abstract syntax trees

An abstract syntax tree represents the essential structure of a program without every punctuation mark or grammar-production detail. An addition node has its operand expressions as children. Parentheses affect the tree's structure but need not remain separate nodes.

Ambiguous expression grammars

E -> E + E | E * E | id is ambiguous because a string such as a + b * c has two different parse trees. One groups the addition first; another groups the multiplication first. A grammar that separates additive and multiplicative expressions can encode precedence.

Precedence and associativity

Precedence determines which operators bind more tightly: multiplication typically binds more tightly than addition. Associativity resolves grouping among operators of the same precedence. Left-associative subtraction groups a - b - c as (a - b) - c.

Syntax and semantics

Syntax specifies the legal form of a program. Semantics specifies what a program means or how it behaves. A syntactically valid expression can still have a type error, and two differently written expressions may have the same meaning.

Original mock course material for a portfolio demo. Not an institutional handout.

Programming languages

05 / Scope, types and functions

Static scope and lexical binding

Static scope, also called lexical scope, resolves a name using the program's nested source-code scopes. A use of x refers to the nearest enclosing declaration visible at that source position, regardless of which function called the current function.

Dynamic scope

Dynamic scope resolves a name by searching active function calls, starting with the current call and proceeding through its callers. The binding can therefore change with the calling sequence. Under static scope, the relevant enclosing scopes come from source structure instead.

Closures

A closure pairs a function with its captured lexical environment. For example, makeAdder(n) returns a function that computes n + x. Calling makeAdder(3) produces an add3 closure that remembers n = 3 even after makeAdder has returned.

Progress and preservation

For a suitable typed language, progress states that a well-typed term is a value or can take a computation step. Preservation states that taking a step preserves its type. Together these properties establish type safety: well-typed evaluation does not become stuck in the errors excluded by the type system.

Capture-avoiding substitution

Substitution replaces free occurrences of a variable while preserving binding relationships. If a replacement would cause a free variable to become bound, first rename the conflicting bound variable. This renaming is alpha-conversion; it avoids variable capture.

Original mock course material for a portfolio demo. Not an institutional handout.